`elements.add`), this could lead to overflow of the cache, or just to one of the two values being overwritten by the other.
It's important to understand that even if the cache handles purging internally, as it should be (the app should only worry about reading/writing elements, only the cache should decide when purging is necessary), if cache's methods are not synchronized, race conditions can and do happen inside those methods.

The issue here is that the set operation is not *atomic*: while we are in the process of setting a new entry (or updating an old one) at the same time we might be performing the same operation in another thread for another entry, the same operation for the same entry, or even an entirely different operation, like evicting an element from the cache.

When we operate in a single-threaded environment, calling cache's methods synchronously, things go smoothly because each method call is completed before anyone else can change the cache, and so we have the illusion of atomicity.

In a multi-threaded environment, instead, we need to be careful and explicitly regulate the execution of these methods and access to the shared resources, so that all methods that mutate the state of an object are executed simulating full isolation (so no one else can modify the resource, and perhaps also no one else can read from it). Figure 7.13 can give you an idea of how we can correct the workflow to prevent the race conditions previously discussed.